Тема 1: Графи. Дървета. Обхождане на графи.

ТЕОРИЯ

1. Дефиниции за граф

Краен ориентиран мултиграф $G = (V, E, \varphi)$, където:

- V — крайно множество от върхове (vertices)
- E — крайно множество от ребра (edges)
- $\varphi : E \to V \times V$ — функция, задаваща наредена двойка (начало, край) за всяко ребро

Краен неориентиран мултиграф $G = (V, E, \varphi)$, където:

- $\varphi : E \to \{\{u, v\} \mid u, v \in V\}$ — функция, задаваща неупорядъчена двойка върхове

  Мулти — допускат се успоредни ребра (между едни и същи върхове) и примки (ребро $u \to u$).

Прост граф — без примки и без успоредни ребра.

---

2. Маршрут, контур, път, цикъл

Маршрут в ориентиран мултиграф: Редица $v_0, e_1, v_1, e_2, v_2, ..., e_k, v_k$, където за всяко i: $\varphi(e_i) = (v_{i-1}, v_i)$.

- Дължина = k (брой ребра)
- **Контур** — маршрут с $v_0 = v_k$

Път в неориентиран мултиграф: Маршрут без повтарящи се върхове ($v_0, v_1, ..., v_k$ са различни).

- **Цикъл** — затворен път: $v_0 = v_k$, останалите върхове са различни

---

3. Свързаност и свързани компоненти

Неориентиран граф е свързан, ако между всеки два върха $u, v \in V$ съществува път.

Свързани компоненти — класовете на еквивалентност по релацията „съществува път между u и v". Всяка компонента е максимален свързан подграф.

---

4. Дърво

Дърво — свързан неориентиран граф без цикли.

Еквивалентни характеризации (всяко едно от следните):

1. Граф без цикли и свързан
2. Между всеки два върха има точно един прост път
3. Свързан граф, в който премахването на произволно ребро го прави несвързан
4. Граф без цикли, в който добавянето на произволно ново ребро създава цикъл
5. Свързан граф с $|V| = |E| + 1$

---

5. Кореново дърво

Кореново дърво (rooted tree) — дефинира се индуктивно:

- **База**: Единичен връх е кореново дърво (той е коренът).
- **Стъпка**: Ако $T_1, T_2, ..., T_k$ ($k \geq 1$) са кореновни дървета с корени $r_1, ..., r_k$, тогава граф с нов корен r, свързан с $r_1, ..., r_k$, е кореново дърво.

Теорема: Всяко кореново дърво е дърво.

Доказателство:

1. **Свързаност**: Всеки връх v е достижим от корена r — по индукция по дълбочината: в индуктивната стъпка v е в някое T_i, а T_i е свързано с r.
2. **Без цикли**: Всеки връх (освен корена) има точно един родител (единственото ребро нагоре по дървото). Ако имаше цикъл, би имало два различни пути от корена до някой връх — противоречие с единствеността на родителя. □

---

6. |V| = |E| + 1 (за дърво)

Теорема: В дърво с n върха има точно $n-1$ ребра.

Доказателство по индукция по n:

База ($n = 1$): 1 връх, 0 ребра. $1 = 0 + 1$. ✓

Индукционна стъпка: Нека твърдението е вярно за всички дървета с $< n$ върха. Разгледаме дърво T с n върха. T е свързано и без цикли $\Longrightarrow$ T има листо (връх от степен 1). Нека l е листо с единственото съседно ребро e.

- $T' = T$ без l и e е свързан граф (l не е мост, защото е листо) без цикли $\Longrightarrow$ T' е дърво с $n-1$ върха.
- По ИХ: T' има $(n-1) - 1 = n-2$ ребра.
- $T = T' + l + e \Longrightarrow$ T има $(n-2) + 1 = n-1$ ребра. □

---

7. Покриващо дърво

Покриващо дърво (spanning tree) на граф G — подграф, който е дърво и съдържа всички върхове на G.

Теорема: Всеки свързан граф има покриващо дърво.

Доказателство: Докато в графа съществува цикъл, премахваме произволно ребро от него. Премахването на ребро от цикъл не нарушава свързаността (другите ребра на цикъла осигуряват алтернативен път). Процесът приключва след краен брой стъпки (|E| намалява) с граф без цикли — т.е. дърво, покриващо всички върхове. □

---

8. Обхождане в ширина (BFS)

```
BFS(G, s):
  за всеки v ≠ s: visited[v] = false, dist[v] = ∞
  visited[s] = true, dist[s] = 0
  Queue Q = {s}
  докато Q не е празна:
    u = dequeue(Q)
    за всяко ребро (u, v):
      ако не visited[v]:
        visited[v] = true
        dist[v] = dist[u] + 1
        parent[v] = u
        enqueue(Q, v)
```

Свойства:

- Обхожда по нива (слоеве)
- Намира **най-кратки пътища** (по брой ребра) от s до всеки достижим връх
- Сложност: O(|V| + |E|)
- Структура данни: **опашка (queue)**

---

9. Обхождане в дълбочина (DFS)

```
DFS(G, s):
  visited[s] = true
  за всяко ребро (s, v):
    ако не visited[v]:
      parent[v] = s
      DFS(G, v)
```

Итеративно — с **стек (stack)**.

Свойства:

- Следва пътя до края, после се връща (backtracking)
- Открива свързани компоненти, топологично сортиране, цикли
- Сложност: O(|V| + |E|)
- Структура данни: **стек (рекурсия или явен стек)**

DFS дърво: ребрата (u, v) за непосетени v образуват дърво (tree edges). Останалите ребра са back edges (към предшественик), forward edges, cross edges.

---

10. Ойлерови обхождания

Ойлеров цикъл (Eulerian circuit) — затворен маршрут, минаващ по всяко ребро **точно веднъж**.

Ойлеров път (Eulerian path/trail) — (не задължително затворен) маршрут, минаващ по всяко ребро точно веднъж.

Теорема за Ойлеров цикъл (с доказателство)

Твърдение: Свързан неориентиран мултиграф G има Ойлеров цикъл ⟺ всеки връх има четна степен.

Необходимост (⟹): Нека C е Ойлеров цикъл. При всяко посещение на връх v, цикълът влиза по едно ребро и излиза по друго — използват се два ребра. Тъй като всяко ребро се използва точно веднъж, степента deg(v) = 2 × (брой посещения) = четно число. □

Достатъчност (⟸): Нека всеки връх има четна степен.

1. Започваме от произволен връх s и строим маршрут W, като при всяка стъпка избираме непосетено ребро. Маршрутът не може да „заседне" в ненасетени ребра в средата (защото степените са четни — влизаш/излизаш по двойки), така че W завършва обратно в s → W е цикъл.
2. Ако W не покрива всички ребра, тъй като G е свързан, съществува връх u ∈ W с непосетени ребра. Строим нов цикъл W' от u по непосетените ребра (пак възможно, защото непосетените ребра имат четни степени).
3. Вмъкваме W' в W при u, получавайки по-дълъг цикъл.
4. Повтаряме докато всички ребра са покрити. □

Теорема за Ойлеров път

Твърдение: Свързан неориентиран мултиграф G има Ойлеров път (не цикъл) ⟺ точно 2 върха имат нечетна степен.

Доказателство: Нека u и v са двата върха с нечетна степен. Добавяме временно ребро (u, v) — сега всички степени са четни ⟹ съществува Ойлеров цикъл (по горната теорема). Премахваме временното ребро от цикъла — получаваме Ойлеров път от u до v. □

> **Следствие**: Ако всички степени са четни → Ойлеров цикъл. Ако точно 2 нечетни → Ойлеров път (не цикъл). Ако > 2 нечетни → няма Ойлерово обхождане.

КАК СЕ РЕШАВАТ ЗАДАЧИ

> Графи и дървета се появяват в практическата част в слота на **Задача 1/2 (C++ код)** — конкретно BFS/DFS обхождане. Не са фиксиран слот в стандартните изпити 2021–2025.

Тип 1: BFS обхождане на дърво/граф — C++ код

Появява се в: 2020 Sep (обхождане на двоично дърво по нива), 2017 Sep (BFS в 2D решетка)

Вариант A — Обхождане по нива на двоично дърво:

```
void printByLevels(Node* root) {
    if (!root) return;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            Node* cur = q.front(); q.pop();
            cout << cur->val << " ";
            if (cur->left)  q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
        cout << "\n";  // ново ниво
    }
}
```

Вариант B — BFS в 2D решетка (намери разстояние):

```
int bfs2D(vector<string>& grid, pair<int,int> start, pair<int,int> end) {
    int dx[] = {0,0,1,-1}, dy[] = {1,-1,0,0};
    int R = grid.size(), C = grid[0].size();
    vector<vector<int>> dist(R, vector<int>(C, -1));
    queue<pair<int,int>> q;
    dist[start.first][start.second] = 0;
    q.push(start);
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int d = 0; d < 4; d++) {
            int nx = x+dx[d], ny = y+dy[d];
            if (nx>=0 && nx<R && ny>=0 && ny<C
                && grid[nx][ny] != '#' && dist[nx][ny] == -1) {
                dist[nx][ny] = dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }
    return dist[end.first][end.second];
}
```

Тип 2: DFS/обхождане на дърво — C++ код

Появява се в: 2019 Sep (обхождане на означено дърво), 2020 задача с рекурсия върху дърво

Намери всички пътища root → листа (DFS с backtracking):

```
void paths(Node* node, vector<int>& path) {
    if (!node) return;
    path.push_back(node->val);
    if (!node->left && !node->right) {
        for (int x : path) cout << x << " ";
        cout << "\n";
    } else {
```

```
        paths(node->left, path);
        paths(node->right, path);
    }
    path.pop_back();  // backtrack
}
```

Бързи факти

Факт	**Стойност**
Дърво с n върха има	n−1 ребра
Ойлеров цикъл съществува ↔	всички степени четни
Ойлеров път (не цикъл) ↔	точно 2 нечетни степени
BFS структура данни	Опашка (Queue)
DFS структура данни	Стек / рекурсия
BFS намира	Най-кратък път (по ребра)